

HackNest Architecture — Index

Document: `docs/architecture/README.md`

System: HackNest — Student Operating System

Status: Foundational — resolves Critical/Medium finding from `ARCHITECTURE_AUDIT_REPORT.md` (Section 1, Item 4)

Type: Index / Orientation Document — No Implementation

Purpose of This Directory

`docs/architecture/` is the authoritative specification set for HackNest. Every document in this directory answers the question "how is this part of the system supposed to work," independent of any specific line of code. Nothing in this directory is implementation — it is the plan that implementation must conform to.

This document is the entry point. It exists so that anyone — engineer, reviewer, or future contributor — can find the right document for the question they have, and can see how the documents relate to and depend on each other, without needing to read all of them first.

This document directly resolves the gap identified in `ARCHITECTURE_AUDIT_REPORT.md` (Section 1: "Missing `README.md`"), which previously left every other document without an authoritative root to orient against.

What HackNest Is

HackNest is a **Student Operating System**: a single, coherent platform where a student's identity, academic affiliation, skills, teams, projects, hackathon participation, and community presence live behind one account. Rather than a single-purpose tool (just a hackathon site, or just a portfolio site, or just a chat app), HackNest is designed as the recurring home base a student returns to throughout their academic and early-builder journey.

The system is built on a modern **Next.js + Supabase + TypeScript** stack, with a server-first application architecture, PostgreSQL-backed data layer with Row Level Security, and a design system oriented around clarity, accessibility, and a dark-first visual identity suited to a builder-focused audience.

Document Index

The table below lists every document expected in this directory, its purpose, and its current status as of this document's authoring.

Document	Purpose	Status
<code>README.md</code> (this document)	Orientation, index, and cross-document map.	Complete
<code>MASTER_ARCHITECTURE.md</code>	The top-level system architecture: product scope, domain ownership boundaries, system-wide terminology, and how all subsystem architectures relate.	Next in rem
<code>MASTER_GOVERNANCE_AMENDMENT_001.md</code>	Defines the governance process for how architecture documents themselves are approved, amended, and enforced — including approval authority for security, release, and design decisions.	Not yet proc
<code>PHASE_TEMPLATE.md</code>	The canonical template for how implementation phases (as used in <code>IMPLEMENTATION_ROADMAP.md</code>) must be structured.	Not yet proc
<code>CODING_STANDARDS.md</code>	Naming, structure, and tooling conventions for the codebase itself.	Not yet proc
<code>DATABASE_ARCHITECTURE.md</code>	The data architecture: domains, relationships, naming conventions, RLS philosophy, storage, migrations, performance, and scalability.	Next in rem after <code>MASTER_AF</code>
<code>API_ARCHITECTURE.md</code>	The API architecture: principles, domain responsibilities, request/response standards, security, performance, and monitoring for all Server Actions and Route Handlers.	Complete (standalone; reconciliatic <code>MASTER_AF</code> and <code>DATABASE_</code> exist)

Document	Purpose	Status
SECURITY_ARCHITECTURE.md	The security architecture: authentication, authorization, database security, secrets management, API/frontend/file-upload security, monitoring, backup, and compliance.	Complete (standalone; reconciliatio remediator
UI_DESIGN_SYSTEM.md	The design system: principles, brand identity, color/typography/spacing usage rules, components, navigation, responsive behavior, accessibility, animation, and page templates.	Complete (standalone; reconciliatio MASTER_AF exists)
TESTING_STRATEGY.md	The testing architecture: principles, testing levels, environments, test data, performance/security/accessibility testing, release quality gates, and CI/CD testing requirements.	Complete (standalone; reconciliatio
DEPLOYMENT_ARCHITECTURE.md	The deployment architecture: system architecture, environments, CI/CD strategy, infrastructure, configuration, monitoring, scaling, backup/disaster recovery, and release strategy.	Complete (standalone; reconciliatio remediator
OPERATIONS_ARCHITECTURE.md	The operations architecture: day-to-day operational ownership, on-call/escalation structure, incident handling in practice, and operational runbooks-at-the-architecture-level (not implementation).	Not yet pro in current re
IMPLEMENTATION_ROADMAP.md	The 27-phase execution plan from project setup to production-ready V1.	Complete; e own depend DATABASE_
ARCHITECTURE_AUDIT_REPORT.md	The audit identifying missing documents, broken/circular phase dependencies, duplicate specifications, and unsafe-to-implement phases.	Complete; t the remedie prescribes : README ar documents

How These Documents Relate

At a high level, the documents form three layers:

1. **Governance layer** — `MASTER_ARCHITECTURE.md` and `MASTER_GOVERNANCE_AMENDMENT_001.md` define what the system is and who has authority to change how it's specified. Everything else answers to this layer.
2. **Subsystem architecture layer** — `DATABASE_ARCHITECTURE.md`, `API_ARCHITECTURE.md`, `SECURITY_ARCHITECTURE.md`, `UI_DESIGN_SYSTEM.md`, `TESTING_STRATEGY.md`, `DEPLOYMENT_ARCHITECTURE.md`, and `OPERATIONS_ARCHITECTURE.md` each own one concern in depth, cross-referencing each other but never duplicating each other's authoritative content (e.g., `SECURITY_ARCHITECTURE.md` describes RLS *philosophy* and cooperation with the API layer, but `DATABASE_ARCHITECTURE.md` alone owns the actual RLS strategy per domain).
3. **Execution layer** — `IMPLEMENTATION_ROADMAP.md` sequences the actual build against everything above it, and `ARCHITECTURE_AUDIT_REPORT.md` (and any future audit) checks that the first two layers are internally consistent before execution proceeds.

A useful rule for avoiding duplicate specifications (a problem the audit report identified): **if two documents seem to need to define the same thing, one of them is wrong to define it — it should reference the other instead.** Section ownership is intentionally exclusive per the table above.

Reading Guide by Role

- **A new engineer** should read this document, then `MASTER_ARCHITECTURE.md`, then the subsystem document(s) relevant to what they're building.
 - **A reviewer approving a security-sensitive change** should read `SECURITY_ARCHITECTURE.md` and the relevant phase in `IMPLEMENTATION_ROADMAP.md`.
 - **Anyone planning a new phase or feature not yet in the roadmap** should check `MASTER_ARCHITECTURE.md` for domain ownership before writing anything new, to avoid duplicating a domain that already has an owner.
 - **Anyone auditing the documentation itself** should follow the process and structure established in `ARCHITECTURE_AUDIT_REPORT.md`.
-

Definition of Done

- [x] Index and orientation only — no implementation, no code, no SQL.
- [x] Every existing and expected document listed with accurate status.
- [x] Cross-document relationship and layering explained.
- [x] Directly resolves the README.md gap identified in ARCHITECTURE_AUDIT_REPORT.md .

Validation

Self-validation against ARCHITECTURE_AUDIT_REPORT.md :

Audit Finding	Resolution in This Document
Section 1, Item 4 — Missing README.md (Medium risk)	Resolved: this document now exists as the authoritative index.
Section 2 — Documents built from assumption chains with no root	Partially addressed: this document establishes the intended root and layering, but full resolution requires MASTER_ARCHITECTURE.md (next in sequence) to actually populate the governance-layer content this document defers to.
Section 6 — Duplicate/overlapping specifications (e.g., audit logging owned by 3 documents)	Partially addressed: the "exclusive section ownership" rule above is introduced as the governing principle; actual reassignment of the audit-logging specification to DATABASE_ARCHITECTURE.md alone will happen when that document is produced next-but-one in this sequence.

Outstanding dependency for full validation: This document references MASTER_ARCHITECTURE.md , DATABASE_ARCHITECTURE.md , MASTER_GOVERNANCE_AMENDMENT_001.md , PHASE_TEMPLATE.md , CODING_STANDARDS.md , and OPERATIONS_ARCHITECTURE.md — none of which exist yet at the moment this document is authored, apart from the remediation sequence itself. This document's content (the index and layering model) does not depend on their contents to be correct, so it is considered **valid and complete on its own terms**, pending the documents it indexes actually being produced per the recommended order.

Status: Validated — Proceeding to MASTER_ARCHITECTURE.md